

AI Audit Report

Student: Nguyen Phan Hung Linh Student ID: 23127081 Session date: 2026-08-09 Timezone: Asia/Ho_Chi_Minh (UTC+07:00)

Declaration

I used AI for these tasks:

- OpenAI Codex agent: read the source code, suggest test cases, write first drafts, and review its drafts.

I checked the requirements, test results, and AI drafts. I fixed the drafts and take responsibility for the final tests. The agent did not edit the files or create the test evidence.

Conversation with the AI

A1 - Source-grounded feature analysis

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 15:43 ICT
- My prompt:

Act as the external AI assistant that do not edit files. First, inspect the local eshop-sut code and identify the exact user-visible behavior, routes, prerequisites, and likely automation risks for FR-03 (forgot/reset password), FR-09 (discount coupons), and FR-12 (admin access control). Return concise findings with source file references. Do not generate scripts yet.

- AI output: The AI listed the UI and API routes, login details, setup needs, and possible risks. It found the four-digit OTP, missing reset controls, weak password check, wrong discount formula, wrong minimum check, missing coupon login check, unsafe user_id, missing admin-role check, and public product changes.
- Human review: I checked the findings in server.js, ForgotPassword.jsx, Checkout.jsx, and the admin application before asking for test cases.

A2 - Design exactly 36 cases

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 15:50 ICT
- My prompt:

Based on your source-grounded analysis, design exactly 12 independent, data-driven cases for each feature FR-03, FR-09, and FR-12 (36 total). For every case give: stable ID, type (positive/negative/edge/security), layer (UI/API), input data, expected result from the written requirements (not the buggy implementation), and whether it likely exposes a current SUT defect. Keep destructive cases minimal and identify setup/cleanup. Also propose separate JSON field schemas for the three datasets. Do not write Playwright code yet.

- AI output: The AI created 12 cases for each feature. It included IDs, expected results, setup, cleanup, likely defects, and JSON fields. The cases covered passwords, OTPs, coupon rules, and admin access.
- Human review: I kept the 36 cases. I replaced the inactive-coupon case, which could not be set up, with an empty-code case. I also split cases that checked two different results.

A3 - Initial FR-03 draft

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 15:55 ICT
- My prompt:

Generate an initial Playwright JavaScript draft for FR-03 only, using the 12-case design and a separate JSON file. Prefer accessible locators and explicit assertions. Tests must express the written requirements, so tests may fail when they reveal SUT defects. Keep tests independent and avoid modifying the SUT. Include comments only where setup or an expected defect is not obvious. Return code blocks for `test-data/fr03-cases.json` and `tests/fr03.spec.js`; do not edit files.

- AI output: The AI returned a JSON file with 12 rows and a Playwright draft. It included account setup, OTP reading, weak-password cases, OTP security, login checks, and cleanup.
- Mistakes found: It used CommonJS in an ES module project, checked too many things in one case, used weak locators, and depended too much on alert text.
- Human changes: I changed the code to ES modules, gave each case one main check, reduced alert-text checks, added counters for test data, and ran the tests.

A4 - Initial FR-09 draft

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 16:03 ICT
- My prompt:

Generate an initial Playwright JavaScript draft for FR-09 only, using the agreed 12 cases and a separate JSON file. Use the checkout UI for the UI-labeled cases and Playwright request for API cases. Express requirement-correct expectations even when the SUT should fail. Avoid raw SQLite access; create users and usage fixtures through available APIs where possible. Return code blocks only; do not edit files.

- AI output: The AI returned coupon data and Playwright code for discount values, invalid coupons, minimum amounts, guests, usage limits, separate users, and fake user IDs.
- Mistakes found: The guest test matched the Login link in the menu. A deleted coupon was not the same as an inactive coupon. Normal requests sent a user ID from the client. One case checked both user limits and percentage math.
- Human changes: I checked the login URL for guests, used an empty-code case, removed `user__id` from normal requests, and tested user isolation separately.

A5 - Initial FR-12 draft

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 16:08 ICT
- My prompt:

Generate an initial Playwright JavaScript draft for FR-12 only, using the agreed 12 cases and a separate JSON file. Include both admin UI login behavior and backend access-control checks. Use unique mutation fixtures and cleanup only what each test creates. Assert requirement-correct 401/403 responses even though the SUT may wrongly return success. Return code blocks only; do not edit files.

- AI output: The AI returned admin-login tests and API tests for an admin, guest, invalid token, and normal user. It saved created IDs for cleanup.
- Mistakes found: It sent a request body even when one was not needed. It mainly checked status codes and used exact alert text.
- Human changes: I only sent a body when needed. I also checked that blocked coupons, products, and categories were not saved.

A6 - Critique after real Chromium execution

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 16:14 ICT

- My prompt:

I adapted your CommonJS drafts to ESM and ran all 12 cases per feature in Chromium. Results: FR-03 7 passed/5 failed; FR-09 7 passed/5 failed; FR-12 5 passed/7 failed. I also found your FR09 guest poll falsely matched the navbar text “Đăng nhập”, and the inactive-coupon case cannot establish `is_active=0` through available APIs. Critique your drafts: list the mistakes or fragile choices you made, why you made them, and concrete fixes I should apply before the three-browser matrix. Keep this concise and do not edit files.

- AI output: The AI accepted the problems in its drafts. These included the module format, combined checks, weak locators, alert text, false guest match, invalid inactive setup, unsafe `user_id`, cleanup problems, extra request data, and missing saved-state checks.
- Human review: I made the needed fixes, restarted the backend with clean seed data, and ran the nine final browser reports.

A7 - Final audit summary

- Tool: OpenAI Codex agent
- Time: 2026-08-09, approximately 16:30 ICT
- My prompt:

Produce a concise chronological audit summary of our six earlier interactions. For each, include the exact purpose, the main output you provided, and the mistakes later identified. Do not claim that your code was used unchanged. End with a short statement distinguishing your contribution from the student's human review and execution evidence. Do not edit files.

- AI output: The AI summarized the first six steps. It stated that it helped with source reading, test design, and first drafts. I reviewed and changed the drafts, ran the tests, found problems, and created the test evidence.

Final responsibility statement

The final files are not raw AI output. I reviewed and changed the AI drafts. The final checks follow the written requirements. The HTML reports, screenshots, traces, Git history, and GitHub Issues come from real test runs and repository work.